

TravelLink Backend — Progress Notes

Project Location

F:\Projects\TravelLink\Server\Server\

Tech Stack

- .NET 8, ASP.NET Core Web API
 - Entity Framework Core + SQL Server (TravelLinkDb)
 - JWT Authentication (Microsoft.AspNetCore.Authentication.JwtBearer v8.0.0)
 - BCrypt.Net-Next (password hashing)
 - CloudinaryDotNet (image storage)
 - SignalR (real-time location)
 - Gemini 1.5 Flash via direct HTTP REST (no SDK — itinerary AI)
 - Open-Meteo API (free weather, no key needed)
-

□ Completed Modules

Entities (Models/Entities/)

- User.cs — includes ImageUrl, CreatedGroups nav property
- FriendRequest.cs
- Group.cs — includes CoverImageUrl
- GroupMember.cs
- Expense.cs
- ExpenseSplit.cs

AppDbContext (Data/AppDbContext.cs)

- All DbSet registered
- Full Fluent API configured
- HasPrecision(18, 2) on all decimal properties (Expense.Amount, ExpenseSplit.AmountOwed, ItineraryRequest.BudgetPerPersonPerDay, ItineraryItem.EstimatedCostPerPerson)

Database

- `appsettings.json` — ConnectionStrings + Jwt + Cloudinary + Gemini sections all correct
- All migrations applied — `TravelLinkDb` live

Common (`Common/ServiceResult.cs`)

- Generic `ServiceResult<T>` with `Success`, `Message`, `Data`
- Static factory methods: `Ok(data)` and `Fail(message)`
- Used by all Expense service methods for structured error messages → toaster UI

Program.cs ☐

- `DbContext`, `JWT`, `CORS`, `Cloudinary Singleton`, `SignalR` all registered
- All services: `IAuthService`, `IFriendService`, `IGroupService`, `IImageService`, `IExpenseService`
- `UseAuthentication()` + `UseAuthorization()` in correct order
- `app.UseCors("auth")` with `WithOrigins("http://localhost:4200")` + `AllowCredentials()` for `SignalR`
- `app.MapHub<LocationHub>("/hubs/location")` registered
- `JWT Events.OnMessageReceived` reads token from `?access_token=` query string for `SignalR WebSocket`

Auth Module ☐

- Register / Login endpoints tested & working

Friend System Module ☐

Endpoint	Route
Send request	POST /api/friend/send
Accept/Reject	POST /api/friend/respond/{id}? action=accept\ reject
Pending requests	GET /api/friend/pending
Friends list	GET /api/friend/list
Unfriend	DELETE /api/friend/unfriend/{targetUserId}

Design decisions:

- Unfriend blocked if unsettled **1-on-1** expenses exist between users

- Group expenses do NOT block unfriending

Group Module ☐

Endpoint	Route
Create group	POST /api/group/create
My groups	GET /api/group/my
Get group by ID	GET /api/group/{id}
Add member (Admin)	POST /api/group/{id}/add-member
Leave group	DELETE /api/group/{id}/leave
Remove member (Admin)	DELETE /api/group/{id}/remove-member
Upload group cover (Admin)	POST /api/group/{id:guid}/cover

Image Module ☐ (Cloudinary)

- IImageService / ImageService (UploadImageAsync, DeleteImageAsync)

User Module ☐

Endpoint	Route
Get profile	GET /api/user/profile
Upload/Update avatar	POST /api/user/avatar

☐ Expense Module — COMPLETE

Design Decisions

- SplitType: "Equal" or "Custom" — validated early, rejects anything else
- Payer NOT force-added to splits — supports "paid for others" scenario
- Warning field in ExpensedTo when payer not in their own split
- Rounding remainder assigned to last participant
- Payer's own split auto-marked IsPaid = true
- Delete blocked if any participant already settled
- Balance netting: eliminates mutual debts
- **Friends-only rule:** 1-on-1 expenses only with accepted friends (group exempt)

Services ☐

- `IService.cs + ExpenseService.cs` — all 8 methods return `ServiceResult<T>`

Controller ☐

Endpoint	Route
Add expense	POST /api/expense/add
Get by ID	GET /api/expense/{id}
Delete	DELETE /api/expense/{id}
Group expenses	GET /api/expense/group/{groupId}
Group balances	GET /api/expense/group/{groupId}/balances
Group analytics	GET /api/expense/group/{groupId}/analytics
Settle split	PUT /api/expense/split/{splitId}/pay
User-to-user summary	GET /api/expense/user/{userId}/summary

☐ Location Sharing Module — COMPLETE (Backend only)

Design Decisions

- Pure relay — NO DB writes per location update (stateless hub)
- Max 5 devices per session
- One active session per user at a time (auto-leaves previous on new join)
- Group session: membership checked against `GroupMembers` table
- 1-on-1 session: friendship checked against `FriendRequests` table
- No DB migration needed

Files Created

- `Dtos/Location/LocationUpdatedDto.cs`
- `Hubs/LocationHub.cs`

Hub Methods

Method	What it does
JoinGroupSession(groupId)	Verify member → auto-leave old → join group room
JoinPrivateSession(targetUserId)	Verify friends → auto-leave old → join private room
SendLocation(dto)	Broadcast to all others in session
LeaveSession()	Voluntarily exit, notify others
OnDisconnectedAsync	Auto-cleanup on crash/browser close

☐ Smart Itinerary Builder — IN PROGRESS

Architecture Decisions

- **Save to DB:** Itineraries persisted — members collaborate async, no re-generation waste
- **Gemini 1.5 Flash via direct HTTP** — no NuGet SDK, full prompt control, ~\$0.0004/generation
- **Open-Meteo** for weather (free, no API key) — geocoding + forecast calls during generation
- **Affiliate links** handled entirely in Angular frontend — backend only stores BookingSearchQuery + BookingType
- **Feasibility filter:** Gemini picks what fits the timeline, returns dropped items with reasons
- **Two-layer priority:** Admin-approved (must include) → vote count → neutral

appsettings.json Gemini section

```
"Gemini": {  
  "ApiKey": "YOUR_KEY_HERE",  
  "Model": "gemini-1.5-flash"  
}
```

☐ Entities Created (Models/Entities/)

File	Purpose
ItineraryRequest.cs	Admin creates one per trip (destination, dates, budget, status)
PlaceSuggestion.cs	One row per suggested place/activity — has AdminApproved (bool?)
SuggestionVote.cs	Upvotes — unique index on (SuggestionId, UserId)
GeneratedItinerary.cs	Gemini output — one-to-one with request, stores DroppedSuggestionsJson

File	Purpose
ItineraryDay.cs	One per day — has WeatherNote
ItineraryItem.cs	Each activity — has IsCompleted, BookingSearchQuery, BookingType

□ AppDbContext Updated

- 6 new DbSet registered
- Full Fluent API for all 6 new entities
- Unique index on SuggestionVote(SuggestionId, UserId)
- One-to-one HasOne...WithOne for GeneratedItinerary ↔ ItineraryRequest

□ Migration Applied

- AddItineraryModule migration created and applied

□ DTOs Created (DTOs/Itinerary/)

File	Direction
CreateItineraryRequestDto.cs	Request — admin creates trip request
AddSuggestionDto.cs	Request — member submits suggestion
ReviewSuggestionDto.cs	Request — admin approves/rejects suggestion
GenerateItineraryDto.cs	Request — optional note for generation/regeneration
SuggestionDto.cs	Response — suggestion + vote count + HasCurrentUserVoted
DroppedSuggestionDto.cs	Response — name + reason (for display)
ItineraryItemDto.cs	Response — one activity
ItineraryDayDto.cs	Response — one day with list of items
ItineraryResultDto.cs	Response — full itinerary + dropped suggestions

□ Remaining — Build Next Session

1. **IItnineraryService.cs** — interface with all method signatures
2. **ItineraryService.cs** — full implementation:
 - CreateRequestAsync — admin creates request, validates group membership
 - AddSuggestionAsync — member adds suggestion, validates group membership
 - DeleteSuggestionAsync — suggester or admin can delete
 - ToggleVoteAsync — upvote/unvote (toggle pattern)

- ReviewSuggestionAsync — admin sets AdminApproved
- GetSuggestionsAsync — returns all suggestions sorted by priority, with vote counts
- GenerateItineraryAsync — **core method**: a. Validate admin b. Fetch weather from Open-Meteo (geocode → forecast) c. Build compact prompt (approved first → by votes → neutral, exclude rejected) d. POST to Gemini REST API with responseMimeType: "application/json" e. Parse JSON → save GeneratedItinerary + ItineraryDay[] + ItineraryItem[] f. Update request status to "Generated"
- RegenerateItineraryAsync — delete old, call generate again with note
- GetItineraryResultAsync — fetch full itinerary with days + items
- ToggleItemCompleteAsync — mark/unmark activity as done

3. **ItineraryController.cs** — 10 endpoints

4. **Program.cs** — register IItineraryService + HttpClient for Gemini calls

Endpoints Planned

Method	Route	Who
POST	/api/itinerary/request	Admin (group)
POST	/api/itinerary/{requestId}/suggest	Any member
DELETE	/api/itinerary/suggestion/{suggestionId}	Suggester or Admin
POST	/api/itinerary/suggestion/{suggestionId}/vote	Any member (toggle)
PATCH	/api/itinerary/suggestion/{suggestionId}/review	Admin
GET	/api/itinerary/{requestId}/suggestions	Any member
POST	/api/itinerary/{requestId}/generate	Admin
GET	/api/itinerary/{requestId}/result	Any member
PATCH	/api/itinerary/item/{itemId}/complete	Any member
POST	/api/itinerary/{requestId}/	Admin

Method	Route	Who
	regenerate	

➔ After Itinerary Module — Angular Frontend

- Auth, Friends, Groups, Expense, Location map (Leaflet), AI Itinerary UI
 - NgRx state management
 - Angular SignalR client (@microsoft/signalr)
 - Leaflet map with live user markers
 - Affiliate booking link generation in BookingLinkService
-

Key Ports

- HTTPS: https://localhost:7150
- HTTP: http://localhost:5283